SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Debugging & Optimisation Task (Low)
Assessment Tool Weightage: 10%

QUESTIONS		
Q.No	**Question**	**Bloom's Level**
1	Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.	BL2 – Understand
2	Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue. Bloom's Level:	BL2 – Understand
3	Correct a faulty SELECT query where column names are misspelled or improperly referenced.	BL1 – Remember
4	Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.	BL3 – Apply
5	Identify and correct a subquery error where multiple values are returned instead of a single value.	BL2 – Understand
6	Debug a VIEW creation statement that contains invalid column references or syntax errors.	BL1 – Remember
7	Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.	BL3 – Apply
8	Identify violations of integrity constraints in a given table and correct them.	BL2 – Understand

9	Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies.	BL3 – Apply
10	Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.	BL3 – Apply

---

Code of Conduct:

I Shaik Ayesha Tabassum(192525074) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action

Signature of the student: S.Ayesha Tabassum

RUBRICS FOR CALCULATION:

Criteria	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Identification	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

1. Identify and correct errors in a given SQL CREATE TABLE statement with incorrect syntax and missing constraints.

Identifying and Correcting Errors in a SQL CREATE TABLE Statement

Incorrect SQL Statement

```
CREATE TABLE Student
(
   Student_ID INT
   Name VARCHAR(50)
   Age INT
   Gender CHAR(1)
   Dept_ID INT
);
```

Errors in the Statement

1. Missing commas (,) between column definitions.
2. No PRIMARY KEY constraint for Student_ID.
3. No NOT NULL constraint for mandatory columns.
4. No FOREIGN KEY constraint for Dept_ID (if it references another table).
5. No CHECK constraint to validate Age.
6. Missing semicolon after each SQL statement (good practice).

Correct SQL Statement

```
CREATE TABLE Student
(
   Student_ID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   Age INT CHECK (Age >= 18),
   Gender CHAR(1) CHECK (Gender IN ('M','F')),
   Dept_ID INT NOT NULL,
   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

Explanation of Corrections

- **Commas added** after each column definition.
- **PRIMARY KEY** ensures Student_ID is unique and cannot be NULL.
- **NOT NULL** ensures mandatory fields (Name, Dept_ID) always contain values.
- **CHECK (Age >= 18)** allows only valid age values.
- **CHECK (Gender IN ('M','F'))** restricts gender values to M or F.
- **FOREIGN KEY** ensures Dept_ID exists in the Department table.

Example Department Table

```
CREATE TABLE Department
(
   Dept_ID      INT      PRIMARY      KEY,
   Dept_Name VARCHAR(50) NOT NULL
);
```

Output Structure

Student Table

Student_ID	Name	Age	Gender	Dept_ID
101	Ayesha	20	F	1
102	Rahul	21	M	2

Department Table

Dept_ID	Dept_Name
1	Computer Science
2	Information Technology

Conclusion

The corrected CREATE TABLE statement follows proper SQL syntax and includes essential constraints such as **PRIMARY KEY**, **NOT NULL**, **CHECK**, and **FOREIGN KEY**, ensuring data integrity, consistency, and valid relationships between tables.

2. Debug an SQL query that incorrectly uses aggregate functions without GROUP BY and fix the issue.

Problem

A table named **Employee** contains the following columns:

- Emp_ID
- Emp_Name
- Department
- Salary

Incorrect SQL Query

```
SELECT Department, AVG(Salary)
FROM Employee;
```

Error

The query selects both:

- Department (a non-aggregate column)
- AVG(Salary) (an aggregate function)

Since Department is not included in a GROUP BY clause, the query will produce an error in most SQL databases.

Typical Error Message:

Column 'Department' is invalid in the select list because it is not contained in either an aggregate function or the GROUP BY clause.

Correct SQL Query

```
SELECT Department, AVG(Salary) AS Average_Salary
FROM Employee
GROUP BY Department;
```

Sample Employee Table

Emp_ID	Emp_Name	Department	Salary
101	Ayesha	CSE	50000
102	Rahul	CSE	60000
103	Priya	ECE	55000
104	Arjun	ECE	65000
105	Neha	IT	70000

Output

Department	Average_Salary
CSE	55000
ECE	60000
IT	70000

Explanation

- AVG(Salary) calculates the average salary.
- GROUP BY Department groups all employees by department.
- The average salary is then calculated separately for each department.

Conclusion

The error occurred because an aggregate function (AVG) was used together with a non-aggregate column (Department) without a GROUP BY clause. Adding GROUP BY Department fixes the query and correctly returns the average salary for each department.

3. Correct a faulty SELECT query where column names are misspelled or improperly referenced

Consider the following **Employee** table:

Emp_ID	Emp_Name	Department	Salary
101	Ayesha	CSE	50000
102	Rahul	IT	60000
103	Priya	ECE	55000

Faulty SQL Query

```
SELECT EmpName, Departmnt, Salery
FROM Employee;
```

Errors in the Query

1. EmpName is incorrect. The correct column name is Emp_Name.
2. Departmnt is misspelled. The correct column name is Department.
3. Salery is misspelled. The correct column name is Salary.

Correct SQL Query

```
SELECT Emp_Name, Department, Salary
FROM Employee;
```

Output

Emp_Name	Department	Salary
Ayesha	CSE	50000
Rahul	IT	60000
Priya	ECE	55000

Another Example (Improper Table Reference)

Incorrect Query

```
SELECT Employee.Name, Employee.Dept
FROM Employee;
```

Errors:

- Name should be Emp_Name.
- Dept should be Department.
-

Correct Query

```
SELECT Employee.Emp_Name, Employee.Department
FROM Employee;
```

Explanation

- Always use the exact column names defined in the table.
- Check spelling carefully.
- If using table names or aliases, ensure the column names belong to that table.
- SQL identifiers must match the table schema.

Conclusion

The faulty SELECT query is corrected by replacing the misspelled or incorrectly referenced column names with the actual column names from the table. This ensures the query executes successfully and returns the expected data.

4. Fix errors in an SQL query using JOIN, where the join condition is missing or incorrect.

Fix Errors in an SQL Query Using JOIN

Problem

Consider the following tables:

Employee

Emp_ID	Emp_Name	Dept_ID
101	Ayesha	1
102	Rahul	2
103	Priya	1

Department

Dept_ID	Dept_Name
1	Computer Science
2	Information Technology

Incorrect SQL Query (Missing JOIN Condition)

```
SELECT Emp_Name, Dept_Name
FROM Employee
JOIN Department;
```

Error

The query does not contain an ON clause to specify how the two tables should be joined.

This may result in:

- A syntax error (in some SQL databases), or
- A Cartesian product (every employee matched with every department), producing incorrect results.

Correct SQL Query

```
SELECT Emp_Name, Dept_Name
FROM Employee
JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;
```

Another Incorrect Query (Wrong JOIN Condition)

```
SELECT Emp_Name, Dept_Name
FROM Employee
JOIN Department
ON Employee.Emp_ID = Department.Dept_ID;
```

Error

The query joins Emp_ID with Dept_ID, which are unrelated columns. This produces incorrect or no matching records.

Correct Query

```
SELECT Emp_Name, Dept_Name
FROM Employee
JOIN Department
ON Employee.Dept_ID = Department.Dept_ID;
```

Output

Emp_Name	Dept_Name
Ayesha	Computer Science
Rahul	Information Technology
Priya	Computer Science

Explanation

- JOIN combines rows from two tables.
- The ON clause specifies the matching condition.
- Employee.Dept_ID = Department.Dept_ID correctly links each employee to their department.
- Using the wrong columns in the ON clause results in incorrect or missing data.

Conclusion

The SQL query is fixed by adding the correct JOIN condition:

ON Employee.Dept_ID = Department.Dept_ID;

This ensures that employees are matched with their respective departments and the query returns accurate results.

5. Identify and correct a subquery error where multiple values are returned instead of a single value.

Problem

Consider the following **Employee** table:

Emp_ID	Emp_Name	Department	Salary
101	Ayesha	CSE	50000
102	Rahul	CSE	60000
103	Priya	ECE	55000
104	Arjun	IT	70000

Incorrect SQL Query

```
SELECT Emp_Name, Salary
FROM Employee
WHERE Salary = (
   SELECT Salary
   FROM Employee
   WHERE Department = 'CSE'
);
```

Error

The subquery:

```
SELECT Salary
FROM Employee
WHERE Department = 'CSE';
```

returns **two values**:

- 50000
- 60000

However, the = operator expects **only one value**, causing an error.

Typical Error Message:

Subquery returned more than one value.

Correct Query (Using IN)

```
SELECT Emp_Name, Salary
FROM Employee
WHERE Salary IN (
 SELECT Salary
```

```
    FROM Employee
    WHERE Department = 'CSE'
);
```

Output

Emp_Name	Salary
Ayesha	50000
Rahul	60000

Alternative Correction (Using an Aggregate Function)

If the intention is to compare with a **single value**, such as the highest salary in the CSE department:

```
SELECT Emp_Name, Salary
FROM Employee
WHERE Salary = (
    SELECT MAX(Salary)
    FROM Employee
    WHERE Department = 'CSE'
);
```

Output

Emp_Name	Salary
Rahul	60000

Explanation

- Use = only when the subquery returns **one value**.
- Use IN when the subquery returns **multiple values**.
- Use aggregate functions like MAX(), MIN(), or AVG() when a single result is required.

Conclusion

The error occurs because the subquery returns multiple rows while the = operator expects a single value. Replacing = with IN (or modifying the subquery to return one value using an aggregate function) resolves the error and produces the correct result.

6. Debug a VIEW creation statement that contains invalid column references or syntax errors.

Problem

Consider the following **Employee** table:

Emp_ID	Emp_Name	Department	Salary
101	Ayesha	CSE	50000
102	Rahul	IT	60000
103	Priya	ECE	55000

Incorrect VIEW Statement

```
CREATE VIEW Employee_View AS
SELECT EmpName, Departmnt, Salary
FROM Employee;
```

Errors

1. EmpName is an invalid column name. The correct name is Emp_Name.
2. Departmnt is misspelled. The correct name is Department.
3. The view cannot be created because the selected column names do not exist in the Employee table.

Correct VIEW Statement

```
CREATE VIEW Employee_View AS
SELECT Emp_Name, Department, Salary
FROM Employee;
```

Another Example (Syntax Error)

Incorrect Query

```
CREATE VIEW Employee_View
SELECT Emp_Name, Department, Salary
FROM Employee;
```

Error

- The keyword AS is missing after the view name.

Correct Query

```
CREATE VIEW Employee_View AS
SELECT Emp_Name, Department, Salary
FROM Employee;
```

Verify the View

SELECT * FROM Employee_View;

Output

Emp_Name	Department	Salary
Ayesha	CSE	50000
Rahul	IT	60000
Priya	ECE	55000

Explanation

- Ensure all column names match the table schema exactly.
- Include the AS keyword when creating a view.
- The SELECT statement inside the view must be syntactically correct.
- Verify that the referenced table and columns exist before creating the view.

Conclusion

The VIEW creation statement is corrected by replacing invalid column names with the correct ones and using the proper SQL syntax (CREATE VIEW view_name AS SELECT ...). This allows the view to be created successfully and queried without errors.

7. Examine an SQL query with unnecessary conditions and optimize it by simplifying the WHERE clause.

Problem

Consider the following **Employee** table:

Emp_ID	Emp_Name	Department	Salary
101	Ayesha	CSE	50000
102	Rahul	IT	60000
103	Priya	ECE	55000
104	Arjun	IT	70000

Inefficient SQL Query

```
SELECT *
FROM Employee
WHERE Salary > 50000
AND Salary >= 50000
AND Department = 'IT'
AND Department = 'IT';
```

Problems in the Query

1. Salary > 50000 and Salary >= 50000 are redundant.
    - Salary > 50000 is more restrictive, so Salary >= 50000 is unnecessary.
2. Department = 'IT' is repeated twice.
3. Extra conditions make the query harder to read and maintain.

Optimized SQL Query

```
SELECT *
FROM Employee
WHERE Salary > 50000
AND Department = 'IT';
```

Output

Emp_ID	Emp_Name	Department	Salary
102	Rahul	IT	60000
104	Arjun	IT	70000

Another Example

Inefficient Query

```
SELECT *
FROM Employee
WHERE (Department = 'CSE' OR Department = 'CSE')
```

```
AND Salary > 40000;
```

Optimized Query

```
SELECT *
FROM Employee
WHERE  Department  =  'CSE'
AND Salary > 40000;
```

Benefits of Optimization

- Removes redundant conditions.
- Improves query readability.
- Reduces unnecessary processing by the database.
- Makes the query easier to maintain and debug.

Conclusion

The query is optimized by removing duplicate and unnecessary conditions from the WHERE clause while preserving the same result. A simplified query is more efficient, readable, and easier to maintain.

8. Identify violations of integrity constraints in a given table and correct them.

Problem

Consider the following **Student** table:

Student_ID	Name	Age	Dept_ID
101	Ayesha	20	1
102	Rahul	-5	2
101	Priya	19	3
NULL	Arjun	21	1
105	Neha	22	10

Assume the following constraints:

- Student_ID → **PRIMARY KEY**
- Age >= 18 → **CHECK** constraint
- Student_ID → **NOT NULL**
- Dept_ID → **FOREIGN KEY** referencing the Department table

Violations

1. Primary Key Violation

- Student_ID = 101 appears twice.
- A primary key must contain **unique** values.

Correction:

```
UPDATE        Student
SET Student_ID = 103
WHERE Name = 'Priya';
```

2. CHECK Constraint Violation

- Rahul has Age = -5.
- Age should be **18 or above**.

Correction:

```
UPDATE Student
SET Age = 18
WHERE Name = 'Rahul';
```

3. NOT NULL Constraint Violation

- Arjun has Student_ID = NULL.
- A primary key cannot be NULL.

Correction:

```
UPDATE        Student
SET Student_ID = 104
WHERE Name = 'Arjun';
```

4. Foreign Key Constraint Violation

Suppose the **Department** table contains only:

Dept_ID	Dept_Name
1	CSE
2	IT
3	ECE

- Neha has Dept_ID = 10, which does not exist in the Department table.

Correction:

```
UPDATE Student
SET Dept_ID = 2
WHERE Name = 'Neha';
```

Corrected Student Table

Student_ID	Name	Age	Dept_ID
101	Ayesha	20	1
102	Rahul	18	2
103	Priya	19	3
104	Arjun	21	1
105	Neha	22	2

Explanation

- **Primary Key** ensures each student has a unique ID.
- **NOT NULL** prevents missing values in mandatory columns.
- **CHECK** validates that age is within the allowed range.
- **FOREIGN KEY** ensures each department ID exists in the Department table, maintaining referential integrity.

Conclusion

The table contained violations of **Primary Key**, **NOT NULL**, **CHECK**, and **Foreign Key** constraints. Correcting these values ensures data integrity, consistency, and compliance with the defined database constraints.

9. Fix a relation that violates 1NF or 2NF by removing repeating groups or partial dependencies

Problem
Consider the following relation:
Student_Course

Student_ID	Student_Name	Course_ID	Course_Name
101	Ayesha	C101, C102	DBMS, Java
102	Rahul	C103	Python
103	Priya	C101, C104	DBMS, C++

1NF Violation

Problem

- Course_ID contains multiple values (C101, C102).
- Course_Name also contains multiple values (DBMS, Java).
- Each cell should contain **only one (atomic) value**.

Convert to 1NF
Split the repeating groups into separate rows.

Student_Course (1NF)

Student_ID	Student_Name	Course_ID	Course_Name
101	Ayesha	C101	DBMS
101	Ayesha	C102	Java
102	Rahul	C103	Python
103	Priya	C101	DBMS
103	Priya	C104	C++

2NF Violation

Assume the composite primary key is:

(Student_ID, Course_ID)

Functional Dependencies:

- (Student_ID, Course_ID) → Student_Name, Course_Name
- Student_ID → Student_Name (Partial Dependency)
- Course_ID → Course_Name (Partial Dependency)

Student_Name depends only on Student_ID, and Course_Name depends only on Course_ID, so the relation violates **2NF**.

Convert to 2NF

Student Table

Student_ID	Student_Name
101	Ayesha
102	Rahul
103	Priya

Course Table

Course_ID	Course_Name
C101	DBMS
C102	Java
C103	Python
C104	C++

Student_Course Table

Student_ID	Course_ID
101	C101
101	C102
102	C103
103	C101
103	C104

Explanation

- **1NF** removes repeating groups and ensures every attribute contains a single value.
- **2NF** removes partial dependencies by placing student and course details into separate tables.
- The Student_Course table now stores only the relationship between students and courses.

Conclusion

The original relation violated **1NF** because it contained repeating groups and **2NF** because of partial dependencies. By decomposing the relation into **Student**, **Course**, and **Student_Course** tables, the database becomes normalized, reduces redundancy, and improves data integrity.

10. Identify redundant attributes in a relation and apply basic normalization (up to 3NF) to improve design.

Problem

Consider the following relation:

Employee

Emp_ID	Emp_Name	Dept_ID	Dept_Name	Manager
101	Ayesha	1	CSE	Dr. Kumar
102	Rahul	2	IT	Dr. Meena
103	Priya	1	CSE	Dr. Kumar

Step 1: Identify Redundant Attributes

The following functional dependencies exist:

- Emp_ID → Emp_Name, Dept_ID
- Dept_ID → Dept_Name, Manager

Redundant Attributes

- Dept_Name
- Manager

These depend on Dept_ID, not directly on Emp_ID. As a result, the department information is repeated for every employee in the same department.

Problems

- Data redundancy
- Update anomalies
- Insert anomalies
- Delete anomalies

Step 2: First Normal Form (1NF)

The relation already satisfies **1NF** because:

- All attributes contain atomic values.
- There are no repeating groups.

Step 3: Second Normal Form (2NF)

Since the primary key is Emp_ID (a single attribute), there are no partial dependencies.

Therefore, the relation is already in **2NF**.

Step 4: Third Normal Form (3NF)

There is a transitive dependency:

Emp_ID → Dept_ID → Dept_Name, Manager

To remove it, decompose the relation.

Employee Table

Emp_ID	Emp_Name	Dept_ID
101	Ayesha	1
102	Rahul	2
103	Priya	1

Department Table

Dept_ID	Dept_Name	Manager
1	CSE	Dr. Kumar
2	IT	Dr. Meena

SQL Statements

Employee Table

```
CREATE TABLE Employee (
   Emp_ID INT PRIMARY KEY,
   Emp_Name VARCHAR(50),
   Dept_ID INT,
   FOREIGN KEY (Dept_ID) REFERENCES Department(Dept_ID)
);
```

Department Table

```
CREATE TABLE Department (
   Dept_ID INT PRIMARY KEY,
   Dept_Name VARCHAR(50),
   Manager VARCHAR(50)
);
```

Benefits of 3NF

- Eliminates redundant data.
- Removes transitive dependencies.

- Reduces update, insert, and delete anomalies.
- Improves data integrity and maintainability.

Conclusion

The attributes **Dept_Name** and **Manager** are redundant because they depend on **Dept_ID** rather than directly on **Emp_ID**. By decomposing the relation into **Employee** and **Department** tables, the design satisfies **Third Normal Form (3NF)**, resulting in a more efficient and consistent database.